

From toy to real: a messy tabular dataset

Until today every dataset was clean, numeric and complete. Real tables have gaps, words where numbers should be, columns on wildly different scales — and occasionally a column that gives the answer away. The model is the easy part. The order of operations is where honest results are won or lost.

Unit 4 · Real data

80 minutes

Lab 14 · a leak-proof pipeline

pandas + scikit-learn

The table

Preparation

Leakage

The pipeline

Lab 14

Before Tuesday

PART 1 · 12 MINUTES

Meet the table

891 real passenger records, and three kinds of trouble.

1

The Titanic passenger list is a real historical record, used in teaching for decades because its problems are real. Each row is a passenger; the target `survived` is 0 or 1. The file ships partly cleaned — duplicate columns removed — because today is about leakage and order of operations, not janitorial work.

Column	Kind	Trouble
pclass, sibsp, parch	numeric, small integers	none
age	numeric	177 missing (20%)
fare	numeric	scale from 0 to over 500, heavily skewed
sex, embarked	categorical words	embarked has 2 missing
deck	categorical	688 missing (77%)
alive	categorical	look at it closely

ALWAYS THE BASELINE FIRST

Predicting *did not survive* for every passenger is right **61.6%** of the time. Any model has to beat that to have learned anything. Write it down before you train a thing.

PART 2 · 20 MINUTES

Three preparation steps, each with a reason

Missing values, categories, scales.

2

Missing values

A network cannot multiply by `NaN`. Three honest options: drop the rows (loses 20% of the data for `age`), drop the column (sensible for `deck` at 77% missing), or **impute** — fill each gap with a typical value such as the median. The median of what? That question is the whole session.

Categories

`sex` is a word. Replacing *male* with 0 and *female* with 1 happens to work for two values, but `embarked` = S, C, Q coded as 0, 1, 2 invents an order and a distance that do not exist. **One-hot encoding** gives each category its own 0/1 column instead: `embarked_S`, `embarked_C`, `embarked_Q`.

Scales

`fare` runs into the hundreds while `parch` is 0 to 6. Session 19 showed what unscaled inputs do to gradient descent. **Standardising** subtracts each column's mean and divides by its standard deviation, so every feature arrives with mean 0 and spread 1.



Every one of these steps learns something from data

The imputer learns a median. The scaler learns a mean and a standard deviation. The encoder learns which categories exist. **Learning from data is fitting** — and anything fitted must only ever see training rows.

PART 3 · 20 MINUTES · COMMITTED ON PURPOSE

Data leakage, and a test score that lies

Two leaks: one loud, one quiet.

3

The loud leak

Use every column as a feature, one-hot encode, split, train a logistic regression, score on the held-out test set:

what the screen shows

TEXT

test accuracy: 1.0

A PERFECT SCORE ON REAL DATA IS A SYMPTOM, NOT A RESULT

Nobody predicts human survival perfectly from a passenger list. Look at the columns again:

`alive` is yes exactly when `survived` is 1. It is the target, written in words. **Target leakage** is any feature that would not exist at the moment the prediction is needed — and it always looks like success.

The quiet leak

Now remove `alive`, but compute the median age and the scaling statistics on *all 891* rows before splitting. The test rows have quietly contributed to the numbers used to prepare the training rows. Measured with 10-fold cross-validation:

Order of operations	Mean accuracy
preprocess, then split	0.7912
split, then preprocess	0.7901

A difference of one thousandth. That is the danger: a quiet leak never looks wrong. On a small dataset, with a feature whose statistics drift over time, or with feature *selection* done on all rows, the same mistake can be worth ten points — and you will not see it until the model meets real users.

THE RULE

Split first. Then fit every preparation step on the training rows only, and apply the already-fitted steps to the test rows. The only reportable number is the one produced that way, even when the leaky one is barely different.

PART 4 · 12 MINUTES

A pipeline makes the rule impossible to break

4

Lab 14's structure

PYTHON

```
prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical),
])
pipe = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=2000))])
pipe.fit(X_train, y_train)  # every median, mean and category learned here —
                             training rows only
print(pipe.score(X_test, y_test))
```

output

OUTPUT

0.771

77.1% against a baseline of 61.6%: modest, honest, and real. The pipeline object carries every fitted step together, so the same preparation is applied at prediction time and cross-validation refits the whole thing inside each fold — the quiet leak cannot happen.



Back to SNIA 2030

Session 3 closed on Côte d'Ivoire's national AI strategy and why this course is taught in Grand-Bassam. Imagine a model for a district health programme or a cocoa cooperative. Which column in that table would be the `alive` — recorded after the outcome, and so useless at the moment of decision? Lab 14's reflection asks you to name one.

LAB 14 · 20 MINUTES IN CLASS

A leak-proof pipeline

5

Lab 14 — `CSC4315_Lab14_A_Leak_Proof_Pipeline`, with the `data/` folder beside it. Baseline, the loud leak, three blanks in the pipeline, the quiet leak.

Before Tuesday

1. **Finish Lab 14**, including its extension B: print the confusion matrix of your pipeline.
2. **Think about this:** a model that is 98% accurate — what would you need to know before being impressed?



Session 21 — Tuesday 17 November, 16:00

Accuracy is not enough: the confusion matrix, precision and recall, ROC curves — and a 98%-accurate model that helps nobody.

